

Music Recommender System — Final Report

Project Overview

This project implements a music recommendation system that predicts user track preferences for a Kaggle competition evaluated on ROC AUC. The data flow involves three files:

`trainItem2.txt` ($\approx 49,000$ users with multi-entity ratings) provides the source features for all models; `testItem2.txt` ($20,000$ users $\times$ 6 candidate tracks each) defines the prediction target; and `test2_new.txt` ($1,000$ users $\times$ 6 candidates with binary labels) provides supervised training signal for the PySpark classifiers. The final submission requirement is an ensemble model that combines multiple base submissions into a single weighted prediction.

Base Models

Forty-five base submissions were generated across two methodological families, all drawing their features from the same `trainItem2.txt` source.

Heuristic feature-weighted scorers (39 submissions). Hand-weighted linear combinations of features from a `collectSignals` function — per-user statistics on each candidate track's album, artist, and genre relationships, with hierarchical fallbacks (e.g., if a user has not rated an album directly, fall back to the average rating on tracks within that album). Variants ranged from artist-only weighting (Kaggle AUC 0.796) through full multi-feature combinations (best heuristic 0.861). These models are unsupervised: no labels are involved, only the training features.

PySpark supervised classifiers (6 submissions). LogisticRegression (0.788), DecisionTreeClassifier (0.854), RandomForestClassifier (0.882), and GBTCClassifier (0.885) trained on the `test2_new` ground truth using the same `collectSignals` feature space. The GBTCClassifier was iteratively improved via 3-fold cross-validation grid search and feature expansion — first adding all 11 genre statistics (14 features total), then adding an ALS-derived artist score as a stacking feature with ALS trained on the full $\approx 3.8\text{M}$ user-artist rating interactions (15 features), and finally adding three user-bias-normalized features such as `track_score - user_mean` (18 features). The best individual model achieved a Kaggle AUC of **0.888**. Cross-validation consistently selected `maxDepth=3` across configurations, indicating a genuinely shallow optimal decision boundary in this feature space.

Ensemble Methodology

The final ensemble was implemented as an analytical least-squares fit that constructs an optimal weighted combination of all base submissions without requiring access to the ground truth. The procedure consists of five steps.

Step 1: Re-encode predictions from $\{0, 1\}$ to $\{-1, +1\}$. Each binary prediction is mapped via $\overline{s_i} = 2 \cdot \hat{s}_i - 1$. Under this encoding, element-wise multiplication of a prediction with the ground

truth produces +1 when they match and -1 when they don't, regardless of class label.

Step 2: Stack submissions into matrix S . With $N = 120,000$ predictions and $K = 45$ base submissions, S has shape $N \times K$, each column being one re-encoded submission.

Step 3: Express the optimal weights via least squares. The optimal weight vector $\mathbf{a}_{LS} = [a_1, \dots, a_K]^T$ minimizes $\|x - Sa\|^2$. The closed-form solution is:

$$\mathbf{a}_{LS} = (S^T S)^{-1} S^T x$$

Step 4: Compute $S^T x$ from Kaggle scores alone. This is the key analytical insight that makes the approach viable. Under $\{-1, +1\}$ encoding, the dot product $s_i^T x$ reduces to:

$$s_i^T x = N \cdot (2 P_i - 1)$$

where P_i is the public Kaggle score of submission i , which equals per-user accuracy under balanced binary predictions. The ground truth x is never needed — only the public leaderboard feedback.

Step 5: Apply weights, sort top-3 per user, and threshold. Compute $s_{ensemble} = S \cdot a_{LS}$. For each user's 6 candidates, the top 3 by score are labeled 1 and the bottom 3 are labeled 0.

Implementation and Result

The ensemble was implemented in NumPy and pandas; the matrix operations are small enough that no Spark distribution is required. The full pipeline runs in under 6 seconds. The implementation loads all 45 CSVs, re-encodes them to $\{-1, +1\}$, builds the $N \times K$ matrix S and the K -length vector P of Kaggle scores, computes $S^T x = N \cdot (2P - 1)$, solves $(S^T S) a = S^T x$ via `numpy.linalg.lstsq` (numerically stable; handles rank-deficient $S^T S$ if needed), and produces continuous ensemble scores $S @ a_{LS}$. The standard top-3-per-user ranking then yields the final binary submission.

Diagnostic output confirmed the method's mathematical soundness: $\text{Rank}(S^T S) = 45 / 45$ (full rank, all submissions linearly independent) and $\text{Condition number} = 5.13 \times 10^3$ (well-conditioned, weights are numerically reliable).

The learned weights are consistent with theoretical expectation. The strongest base models received the largest positive weights: the top GBT submission at +0.222, the RandomForest at +0.152, and the strongest heuristic at +0.126. The two below-random submissions (Kaggle AUC 0.311 and 0.331) received small negative weights (-0.073 and -0.046), correctly inverting their anti-correlated votes. Several mid-tier submissions also received small negative weights — this is the $(S^T S)^{-1}$ factor performing its intended redundancy correction by subtracting submissions whose predictions are largely captured by other members of the set.

Final Kaggle public AUC: 0.893. This exceeds the best individual base model (0.888) by +0.005.

Key Learnings

The analytical formula handles inter-model correlation automatically. Earlier attempts at ensembling — raw-AUC-weighted sums, threshold-pruned subsets, and hand-crafted diversity weighting — all underperformed the best individual model. These hand-crafted approaches lacked the principled inverse-covariance correction provided by the least-squares solution. The $(S^T S)^{-1}$ factor captures pairwise correlation between base submissions and assigns weights that exploit unique signal while canceling redundant agreement. No hand-crafted weighting scheme can reproduce this systematically.

The Kaggle public score serves as a proxy for the ground truth. The elegance of this formulation is that the leaderboard feedback (P_i) , normally treated only as an evaluation metric, contains enough information to fit the optimal ensemble weights through the identity $\frac{S_i^T x}{N \cdot (2P_i - 1)} =$. This avoids any need for held-out labels and avoids overfitting to test2_new specifically. The re-encoding from $\{0, 1\}$ to $\{-1, +1\}$ is what makes this identity work; without that step, the dot product does not separate correct from incorrect predictions cleanly.

Negative weights on strong submissions are not a contradiction. When two submissions make highly correlated predictions and one is slightly stronger, the optimal least-squares solution often subtracts the weaker one from the stronger one to isolate unique signal. The negative weight is not a judgment of submission quality; it is a recognition that the submission's signal is largely already represented by other members of the ensemble.

Feature stacking on a single supervised model produced more gain than algorithmic variation. Across the four supervised classifiers, Kaggle AUC spread from 0.788 to 0.885 — bounded by the classifier choice. The single largest gain on the GBT model came from stacking an ALS-derived score as an additional feature (+0.003 Kaggle AUC), demonstrating that feeding richer signals to one model can outperform searching across model families.

Conclusion

The project produced a final Kaggle public AUC of **0.893** via an analytical least-squares ensemble of 45 base submissions. The best individual model — a GBT classifier with 18 features including ALS-derived stacking signals and user-bias normalization — achieved 0.888. The +0.005 ensemble improvement validates the analytical approach: principled mathematical weighting via $a_{LS} = (S^T S)^{-1} S^T x$ is more effective than hand-crafted weighting schemes because it automatically handles the inter-model correlation that bounds simpler ensemble methods.